

Multi – level linked list

–Delete At Beginning

–Space Complexity(Main List) and Algorithm

Program: –

```
void deleteAtBeginning()
{
    if (head == nullptr)
    {
        cout << "List is empty." << endl;
        return;
    }
    Node *temp = head;
    head = head->next;
    // Note: In a full implementation, we should also free
temp->child recursively
    free(temp);
    cout << "Deleted from beginning." << endl;
}

int main()
{
    int choice;
    while (true)
    {
        cout << "5. Delete at Beginning (Main)" << endl;
        cin >> choice;

        switch (choice)
        {
            case 5:
                deleteAtBeginning();
                break;
            default:
                cout << "Invalid choice" << endl;
        }
    }
    return 0;
}
```

Space Complexity

1. Input Space Complexity

Input Space Complexity means: memory occupied by input data used by the function.

There are TWO common academic interpretations

A. Parameter – only Input Space Complexity

This approach considers ONLY:

- *function parameters.*

Here function:

```
void deleteAtBeginning()
```

has: NO parameters.

So:

Parameter – only Input Space Complexity

$O(1)$

because:

- *no input argument is passed,*
- *no parameter memory depends on input size.*

B. Full Data Input/Holistic

Input Space Complexity

This approach considers:

- *entire data structure being processed,*
- *total input data already existing in memory.*

Here the function operates on:

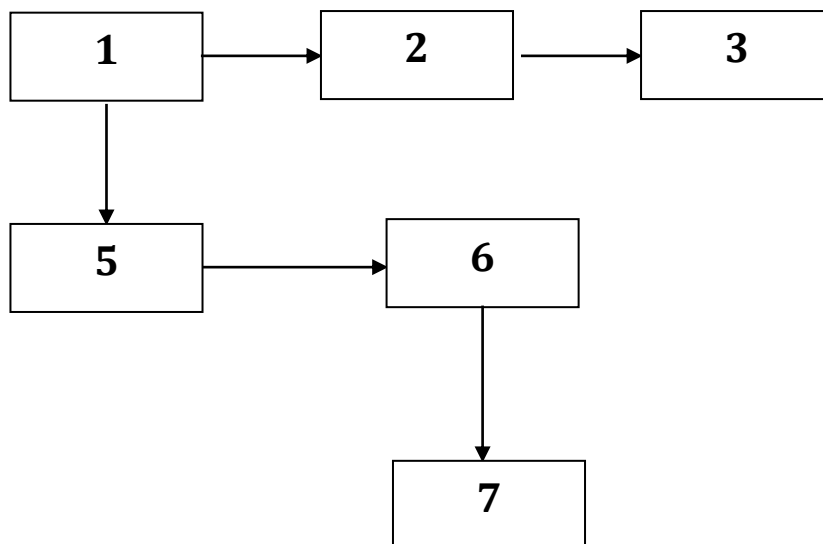
`head`

which may point to a multilevel linked list containing:

- *main nodes,*
- *child nodes,*
- *nested child lists.*

Suppose total nodes in structure = N .

Example:



Total nodes: $N = 7$

The function accesses the multilevel list through:

head

Therefore:

Full Data Input Space Complexity

$$O(N)$$

where:

- *N = total number of nodes existing in the multilevel linked list.*

Important Clarification

Even though the function deletes only ONE node:

```
free(temp);
```

the holistic approach counts: the complete input structure available to the algorithm, not just modified nodes.

```
struct Node
{
    int data;
    struct Node *next; // Points to the right (Same Level)
    struct Node *child; // Points down (Sub Level)
};
```

This belongs to Holistic/ Full Data Input help to create N nodes .

2. Auxiliary Space Complexity

Auxiliary Space means: extra temporary memory used during execution, excluding input data itself.

Memory Used Inside Function

The function creates only:

```
Node *temp
```

This is:

- *one pointer variable,*
- *fixed size,*
- *independent of list size.*

No:

- *arrays,*
- *recursion,*
- *dynamic allocation,*
- *stack growth,*
- *extra linked lists.*

Therefore

Auxiliary Space Complexity

$O(1)$

(Constant extra memory)

Why Not $O(N)$?

Because the function does NOT:

- *traverse recursively,*
- *create additional nodes,*
- *store nodes in arrays/stacks.*

Only:

```
Node *temp
```

is extra memory.

Total Space Complexity

Total Space Complexity = Auxiliary Space + Input Space

Based on the definition of input space you use, the total space complexity changes.

- **Common Interpretation (Parameter-only input):**

$$\text{Space} = \underbrace{O(1)}_{\substack{\downarrow \\ \text{Auxiliary}}} + \underbrace{O(1)}_{\substack{\downarrow \\ \text{Input}}} = O(1).$$

- **Holistic Interpretation (Full-data input):**

$$\text{Space} = \underbrace{O(1)}_{\substack{\downarrow \\ \text{Auxiliary}}} + \underbrace{O(n)}_{\substack{\downarrow \\ \text{Input}}} = O(n).$$

1) If Asked "Total Space Complexity" then natural answer is
: $O(n)$ [Full Data + Auxiliary]

2) If not asked total space complexity then:

In most academic and interview contexts, when asked for the space complexity of a function, the **auxiliary space complexity ($O(1)$)** is the expected answer.

In most academic DSA contexts and coding interviews, when someone simply asks:

“What is the space complexity?”

they usually mean:

Auxiliary Space Complexity

unless they explicitly say:

- total space complexity
- overall memory usage
- data structure space
- input space
- storage complexity

So for your insertion function, the expected answer is usually:

$O(1)$

because only constant extra working memory is used.

Algorithm:

MultiDelAtBegin():

1. [*CHECK IF LIST IS EMPTY*]

If START = NULL, then:

Write: "List is empty."

Return and EXIT.

[End of If structure]

2. [*INITIALIZE TEMPORARY POINTER*]

Set TEMPPTR := START $\left(\begin{array}{l} \text{Point to the starting node to free} \\ \text{its memory} \end{array} \right)$

3. [*Point Head to Next Node*]

SET START := LINK[START]

4. [*Free Beginning Node*]

Call FREE(TEMPPTR)

5. *Write: ``Deleted from Beginning.*
``

6. [*END OF ALGORITHM*]
